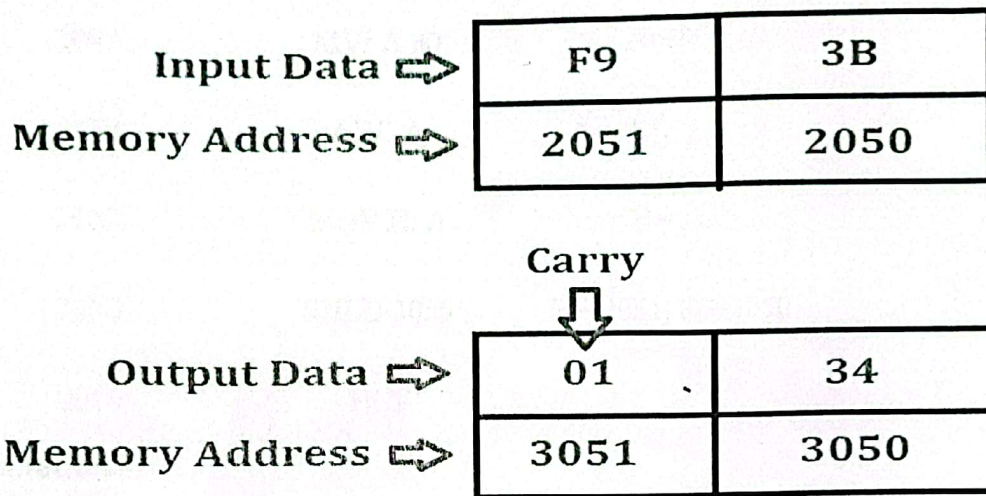


Problem – Write an assembly language program to add two 8 bit numbers stored at address 2050 and address 2051 in 8085 microprocessor. The starting address of the program is taken as 2000.

Example –



Algorithm –

1. Load the first number from memory location 2050 to accumulator.
2. Move the content of accumulator to register H.
3. Load the second number from memory location 2051 to accumulator.
4. Then add the content of register H and accumulator using "ADD" instruction and storing result at 3050
5. The carry generated is recovered using "ADC" command and is stored at memory location 3051

Program –

Memory Address	Mnemonics	Comment
2000	LDA 2050	A<-[2050]
2003	MOV H, A	H<-A
2004	LDA 2051	A<-[2051]

Memory Address	Mnemonics	Comment
2007	ADD H	$A \leftarrow A + H$
2008	MOV L, A	$L \leftarrow A$
2009	MVI A 00	$A \leftarrow 00$
200B	ADC A	$A \leftarrow A + A + \text{carry}$
200C	MOV H, A	$H \leftarrow A$
200D	SHLD 3050	$H \rightarrow 3051, L \rightarrow 3050$
2010	HLT	

Explanation –

1. **LDA 2050** moves the contents of 2050 memory location to the accumulator.
2. **MOV H, A** copies contents of Accumulator to register H to A
3. **LDA 2051** moves the contents of 2051 memory location to the accumulator.
4. **ADD H** adds contents of A (Accumulator) and H register (F9). The result is stored in A itself. **For all arithmetic instructions A is by default an operand and A stores the result as well**
5. **MOV L, A** copies contents of A (34) to L
6. **MVI A 00** moves immediate data (i.e., 00) to A
7. **ADC A** adds contents of A(00), contents of register specified (i.e A) and carry (1). As ADC is also an arithmetic operation, A is by default an operand and A stores the result as well
8. **MOV H, A** copies contents of A (01) to H
9. **SHLD 3050** moves the contents of L register (34) in 3050 memory location and contents of H register (01) in 3051 memory location
10. **HLT** stops executing the program and halts any further execution

Add two 8-bit numbers

Example

(2501 H) = 99H

(2502 H) = 39H

Result (2503 H) = 99H + 39H = D2H

Since,

1	0	0	1	1	0	0	1	(99H)
+	0	0	1	1	1	0	0	1 (39H)
1	1	0	1	0	0	1	0	(D2H)

Program

1. LXI H, 2501H : "Get address of first number in H-L pair. Now H-L points to 2501H"
2. MOV A, M : "Get first operand in accumulator"
3. INX H : "Increment content of H-L pair. Now, H-L points 2502H"
4. ADD M : "Add first and second operand"
5. INX H : "H-L points 4002H"
6. MOV M, A : "Store result at 2503H"
7. HLT : "Stop"

Subtract two 8-bit numbers

Example

(2501 H) = 49H

(2502 H) = 32H

Result (2503 H) = 49H - 32H = 17H

Program

1. LXI H, 2501H : "Get address of first number in H-L pair. Now H-L points to 2501H"
2. MOV A, M : "Get first operand in accumulator"
3. INX H : "Increment content of H-L pair. Now, H-L points 2502H"
4. SUB M : "Subtract first to second operand"
5. INX H : "H-L points 4002H"
6. MOV M, A : "Store result at 2503H"
7. HLT : "Stop"

Add two 16-bits numbers

Add the 16-bit number in memory locations 2501H and 2502H to the 16-bit number in memory locations 2503H and 2504H. The most significant eight bits of the two numbers to be added are in memory locations 2502H and 4004H. Store the result in memory locations 2505H and 2506H with the most significant byte in memory location 2506H.

Example

(2501H) = 15H

(2502H) = 1CH

(2503H) = B7H

(2504H) = 5AH

Result = 1C15 + 5AB7H = 76CCH

(2505H) = CCH

(2506H) = 76H

Program

Add two 16-bits number with ADD and ADC instruction

1. LHLD 2501H : "Get 1st 16-bit number in H-L pair"
2. XCHG : "Save 1st 16-bit number in DE"
3. LHLD 2503H : "Get 2nd 16-bit number in H-L pair"
4. MOV A, E : "Get lower byte of the 1st number"
5. ADD L : "Add lower byte of the 2nd number"
6. MOV L, A : "Store result in L-register"
7. MOV A, D : "Get higher byte of the 1st number"
8. ADC H : "Add higher byte of the 2nd number with CARRY"
9. MOV H, A : "Store result in H-register"
10. SHLD 4004H : "Store 16-bit result in memory locations 2505H and 2506H"
11. HLT : "Stop"

Add two 16-bits numbers with DAD instruction

1. LHLD 2501H : "Get 1st 16-bit number"
2. XCHG : "Save 1st 16-bit number in DE"
3. LHLD 2503H : "Get 2nd 16-bit number in H-L"
4. DAD D : "Add DE and HL"

5. SHLD 2505H : "Store 16-bit result in memory locations 2505H and 2506H".
6. HLT : "Stop"

Subtract two 16-bit numbers

Example

(2500H) = 19H
 (2501H) = 6AH
 (2504H) = 15H (2503H) = 5CH
 Result = 6A19H - 5C15H = 0E04H
 (2504H) = 04H
 (2505H) = 0EH

Program

1. LHLD 2500H : "Get first 16-bit number in HL"
2. XCHG : "Save first 16-bit number in DE"
3. LHLD 2502H : "Get second 16-bit number in HL"
4. MOV A, E : "Get lower byte of the first number"
5. SUB L : "Subtract lower byte of the second number"
6. MOV L, A : "Store the result in L register"
7. MOV A, D : "Get higher byte of the first number"
8. SBB H : "Subtract higher byte of second number with borrow"
9. MOV H, A : "Store 16-bit result in memory locations 2504H and 2505H"
10. SHLD 2504H : "Store 16-bit result in memory locations 2504H and 2505H"
11. HLT : "Terminate program execution"

Add contents of two memory locations

Example

(2500H) = 7FH
 (2501H) = 89H
 Result = 7FH + 89H = 108H
 (2502H) = 08H
 (2503H) = 01H

Program

1. LXI H, 2500H : "HL Points 2500H"

2's complement of a number is obtained by adding 1 to the 1's complement of the number.

Example

To find the two's complement of 96.

96 = 1001 0110

1's complement = 0110 1001 = 69

+ 0000 0001

2's complement = 0110 1010 = 6A

Program

The number is placed in the memory location 2501 H.

The result is to be stored in the memory location 2502 H.

1. LDA 2501 H : "Get data in accumulator"
2. CMA : "Take its 1's complement"
3. ADI, 01 H : "Add one in the number"
4. STA 2502 H : "Store the result in 2502 H"
5. HLT : "Stop"

Count number of 1's in a number

Example

2501 H = 04

2502 H = 34 H

2503 H = A9H

2504 H = 78H

2505 H = 56H

Result = 2503 H = A9H

Program

Count number of 1's of the content of the register D and store the count in the register B.

1. MVI B, 00H
2. MVI C, 08H
3. MOV A, D
4. BACK: RAR

5. JNC SKIP
6. INR B
7. SKIP: DCR C
8. JNZ BACK
9. HLT

Find larger of two numbers

Example

2501H = 98 H
 2502H = 87H
 Result = 98H (2503H)

Program

The first number 98H is placed in the memory location 2501 H.
 The second number 87H is placed in the memory location 2502H.
 The result is stored in the memory location 2503 H.

```
LXI H, 2501H : "Address of first number in H-L pair"
MOV A, M      : "1st number in accumulator"
INX H         : "Address of 2nd number in H-L pair"
CMP M         : "compare 2nd number with 1st number"
JNC AHEAD     : "No, larger is in accumulator. Go to AHEAD"
MOV A, M      : "Yes, get 2nd number in the accumulator"
STA 2503 H    : "Store larger number in 2503H"
HLT           : "Stop"
```

Find smaller of two numbers

Example

2501H = 84 H
 2502H = 99 H
 Result = 84 H (2503H)

Program

The first number 84H is placed in the memory location 2501 H.
 The second number 99H is placed in the memory location 2502H.
 The result is stored in the memory location 2503 H.


```

LXI H, 2501H : "Address of first number in H-L pair"
MOV A, M      : "1st number in accumulator"
INX H         : "Address of 2nd number in H-L pair"
CMP M         : "compare 2nd number with 1st number"
JC AHEAD      : "Yes, smaller number is in accumulator. Go to AHEAD"
MOV A, M      : "No, get 2nd number in the accumulator"
STA 2503 H    : "Store smaller number in 2503H"
HLT           : "Stop"

```

Calculate the sum of series of even numbers

Example

```

2500 H = 4H
2501 H = 20H
2502 H = 15H
2503 H = 13H
2504 H = 22H
Result = 2505 H = 20+22= 42H

```

Program

The numbers are placed in the memory locations 2501 to 2504H.

The sum is to be stored in the memory location 2450H.

As there are 4 numbers in the series, count = 04

The initial value of the sum is made 00. The even number of the series are taken one by one and added to the sum.

1. LDA 2500H
2. MOV C, A : "Initialize counter"
3. MVI B, 00H : "sum = 0"
4. LXI H, 2501H : "Initialize pointer"
5. BACK: MOV A, M : "Get the number"
6. ANI 01H : "Mask Bit 1 to Bit7"
7. JNZ SKIP : "Don't add if number is ODD"
8. MOV A, B : "Get the sum"
9. ADD M : "SUM = SUM + data"
10. MOV B, A : "Store result in B register"
11. SKIP: INX H : "increment pointer"

- 12. DCR C : "Decrement counter"
- 13. JNZ BACK : "if counter 0 repeat"
- 14. STA 2505H : "store sum"
- 15. HLT : "Stop"

Calculate the sum of series of odd numbers

Example

2500 H = 4H

2501 H = 9AH

2502 H = 52H

2503 H = 89H

2504 H = 3FH

Result = 2505 H = 89H + 3FH = C8H

Program

The numbers are placed in the memory locations 2501 to 2504H.

The sum is to be stored in the memory location 2450H.

As there are 4 numbers in the series, count = 04

The initial value of the sum is made 00. The odd number of the series are taken one by one and added to the sum.

- 1. LDA 2500H
- 2. MOV C, A : "Initialize counter"
- 3. LXI H, 2501H : "Initialize pointer"
- 4. MVI E, 00 : "Sum low = 0"
- 5. MOV D, E : "Sum high = 0"
- 6. BACK: MOV A, M : "Get the number"
- 7. ANI 01H : "Mask Bit 1 to Bit-7"
- 8. JZ SKIP : "Don't add if number is even"
- 9. MOV A, E : "Get the lower byte of sum"
- 10. ADD M : "Sum = sum + data"
- 11. MOV E, A : "Store result in E register"
- 12. JNC SKIP
- 13. INR D : "Add carry to MSB of SUM"

14. SKIP: INX H : "Increment pointer"

Find the square of given number

Program

Find the square of 07 (decimal) using lookup table techniques.

The number 07 D is in the memory.

The result is to be stored in the memory location 2501H.

The table for square is stored from 2600 to 2609 H.

1. LDA 2500H : "Get data in accumulator"
2. MOV L, A : "Get data in register L"
3. MVI H, 26H : "Get 26 in register H"
4. MOV A, M : "Square of data in accumulator"
5. STA 2501 H : "Store Square in 2501 H"
6. HLT : "Stop"

Separate even numbers from given numbers

Program

Let's see the program to separate even numbers from the given list of 50 numbers and store them in other location starting from 2600H.

Starting location of 50 number list is 2500H.

1. LXI H, 2500H : "Initialize memory pointer 1"
2. LXI D, 2600H : "Initialize memory pointer2"
3. MVI C, 32H : "Initialize counter"
4. BACK:MOV A, M : "Get the number"
5. ANI 01H : "Check for even number"
6. JNZ SKIP : "If ODD, don't store"
7. MOV A, M : "Get the number"
8. STAX D : "Store the number in result list"
9. INX D : "Increment pointer 2"
10. SKIP: INX H : "Increment pointer 1"

11. DCR C	: "Decrement counter"
12. JNZ BACK	: "If not zero, repeat"
13. HLT	: "Stop"